IND320 project work – Part 3

This notebook samples NVE electricity regions, downloads ERA5 hourly weather from Open-Meteo, detects precipitation outliers with DCT/SPC and temperature anomalies with LOF, and analyses reservoir filling with STL and a spectrogram.

Public links: GitHub repository · Streamlit app

In [10]:
```
from pathlib import Path
import sys
import matplotlib.pyplot as plt
import pandas as pd

PROJECT_ROOT = Path.cwd()
if not (PROJECT_ROOT / 'src' / 'ind320_app').exists():
    raise RuntimeError('Run this notebook from the ind320-app folder.')
sys.path.insert(0, str(PROJECT_ROOT / 'src'))

from ind320_app.data import load_reservoir_data
from ind320_app.part3 import (
    download_weather, plot_lof_temperature, plot_spc_precipitation,
    plot_spectrogram, plot_stl, sample_region_coordinates,
)

SHAPEFILE = PROJECT_ROOT / 'data' / 'NVE_Region.shp'
```

1. Sample 1000 locations per NVE region

The source polygons may use different names for combined areas. The shared function preserves the names from the downloaded shapefile and transforms coordinates to WGS84 before returning longitude/latitude pairs.

In [11]:
```
if not SHAPEFILE.exists():
    raise FileNotFoundError('Download NVE_Region.shp, .shx, .dbf and .prj into data/.')
coordinates = sample_region_coordinates(SHAPEFILE, points_per_region=1000, random_state=42)
print({region: len(points) for region, points in coordinates.items()})
```

```
{'RV': 1000, 'RM': 1000, 'RN': 1000, 'RÃ\x98': 1000, 'RS': 1000}
```

In [12]:
```
# Brukes til praktisk API-testing
coordinates_test = {
    region: points[:10]
    for region, points in coordinates.items()
}
```

2. ERA5 weather data for NO3 in 2024

Open-Meteo supports batched coordinates. The function sends 10 sampled locations per request, caches completed batches and averages the hourly precipitation, temperature and wind speed across all 1000 sampled locations.

In [13]:
```
import importlib
import ind320_app.part3 as part3

importlib.reload(part3)
download_weather = part3.download_weather
```

In [14]:
```
weather_no3 = download_weather(
    'NO3', '2024-01-01', '2024-12-31', coordinates,
    batch_size=10,
    request_pause=2.0,
    cache_dir=PROJECT_ROOT / 'data' / 'weather_cache',
    show_progress=True,
)
display(weather_no3.head())
print('Hourly observations:', len(weather_no3))
```

```
Weather download: batch 1/100 (  1.0%) | points 10/1000 | cache
Weather download: batch 2/100 (  2.0%) | points 20/1000 | cache
Weather download: batch 3/100 (  3.0%) | points 30/1000 | cache
Weather download: batch 4/100 (  4.0%) | points 40/1000 | cache
Weather download: batch 5/100 (  5.0%) | points 50/1000 | cache
Weather download: batch 6/100 (  6.0%) | points 60/1000 | cache
Weather download: batch 7/100 (  7.0%) | points 70/1000 | cache
Weather download: batch 8/100 (  8.0%) | points 80/1000 | cache
Weather download: batch 9/100 (  9.0%) | points 90/1000 | cache
Weather download: batch 10/100 ( 10.0%) | points 100/1000 | cache
Weather download: batch 11/100 ( 11.0%) | points 110/1000 | cache
Weather download: batch 12/100 ( 12.0%) | points 120/1000 | cache
Weather download: batch 13/100 ( 13.0%) | points 130/1000 | cache
Weather download: batch 14/100 ( 14.0%) | points 140/1000 | cache
Weather download: batch 15/100 ( 15.0%) | points 150/1000 | cache
Weather download: batch 16/100 ( 16.0%) | points 160/1000 | cache
Weather download: batch 17/100 ( 17.0%) | points 170/1000 | cache
Weather download: batch 18/100 ( 18.0%) | points 180/1000 | cache
Weather download: batch 19/100 ( 19.0%) | points 190/1000 | cache
Weather download: batch 20/100 ( 20.0%) | points 200/1000 | cache
Weather download: batch 21/100 ( 21.0%) | points 210/1000 | cache
Weather download: batch 22/100 ( 22.0%) | points 220/1000 | cache
Weather download: batch 23/100 ( 23.0%) | points 230/1000 | cache
Weather download: batch 24/100 ( 24.0%) | points 240/1000 | cache
Weather download: batch 25/100 ( 25.0%) | points 250/1000 | cache
Weather download: batch 26/100 ( 26.0%) | points 260/1000 | cache
Weather download: batch 27/100 ( 27.0%) | points 270/1000 | cache
Weather download: batch 28/100 ( 28.0%) | points 280/1000 | cache
Weather download: batch 29/100 ( 29.0%) | points 290/1000 | cache
Weather download: batch 30/100 ( 30.0%) | points 300/1000 | cache
Weather download: batch 31/100 ( 31.0%) | points 310/1000 | cache
Weather download: batch 32/100 ( 32.0%) | points 320/1000 | cache
Weather download: batch 33/100 ( 33.0%) | points 330/1000 | cache
Weather download: batch 34/100 ( 34.0%) | points 340/1000 | cache
Weather download: batch 35/100 ( 35.0%) | points 350/1000 | cache
Weather download: batch 36/100 ( 36.0%) | points 360/1000 | cache
Weather download: batch 37/100 ( 37.0%) | points 370/1000 | cache
Weather download: batch 38/100 ( 38.0%) | points 380/1000 | cache
Weather download: batch 39/100 ( 39.0%) | points 390/1000 | cache
Weather download: batch 40/100 ( 40.0%) | points 400/1000 | cache
Weather download: batch 41/100 ( 41.0%) | points 410/1000 | cache
Weather download: batch 42/100 ( 42.0%) | points 420/1000 | cache
Weather download: batch 43/100 ( 43.0%) | points 430/1000 | cache
Weather download: batch 44/100 ( 44.0%) | points 440/1000 | cache
Weather download: batch 45/100 ( 45.0%) | points 450/1000 | cache
Weather download: batch 46/100 ( 46.0%) | points 460/1000 | cache
Weather download: batch 47/100 ( 47.0%) | points 470/1000 | cache
Weather download: batch 48/100 ( 48.0%) | points 480/1000 | cache
Weather download: batch 49/100 ( 49.0%) | points 490/1000 | cache
Weather download: batch 50/100 ( 50.0%) | points 500/1000 | cache
Weather download: batch 51/100 ( 51.0%) | points 510/1000 | cache
Weather download: batch 52/100 ( 52.0%) | points 520/1000 | cache
Weather download: batch 53/100 ( 53.0%) | points 530/1000 | cache
Weather download: batch 54/100 ( 54.0%) | points 540/1000 | cache
Weather download: batch 55/100 ( 55.0%) | points 550/1000 | cache
Weather download: batch 56/100 ( 56.0%) | points 560/1000 | cache
Weather download: batch 57/100 ( 57.0%) | points 570/1000 | cache
Weather download: batch 58/100 ( 58.0%) | points 580/1000 | cache
Weather download: batch 59/100 ( 59.0%) | points 590/1000 | cache
Weather download: batch 60/100 ( 60.0%) | points 600/1000 | cache
Weather download: batch 61/100 ( 61.0%) | points 610/1000 | cache
Weather download: batch 62/100 ( 62.0%) | points 620/1000 | cache
Weather download: batch 63/100 ( 63.0%) | points 630/1000 | cache
Weather download: batch 64/100 ( 64.0%) | points 640/1000 | cache
Weather download: batch 65/100 ( 65.0%) | points 650/1000 | cache
Weather download: batch 66/100 ( 66.0%) | points 660/1000 | cache
Weather download: batch 67/100 ( 67.0%) | points 670/1000 | cache
Weather download: batch 68/100 ( 68.0%) | points 680/1000 | cache
Weather download: batch 69/100 ( 69.0%) | points 690/1000 | cache
Weather download: batch 70/100 ( 70.0%) | points 700/1000 | cache
Weather download: batch 71/100 ( 71.0%) | points 710/1000 | cache
Weather download: batch 72/100 ( 72.0%) | points 720/1000 | cache
Weather download: batch 73/100 ( 73.0%) | points 730/1000 | cache
Weather download: batch 74/100 ( 74.0%) | points 740/1000 | cache
Weather download: batch 75/100 ( 75.0%) | points 750/1000 | cache
Weather download: batch 76/100 ( 76.0%) | points 760/1000 | cache
Weather download: batch 77/100 ( 77.0%) | points 770/1000 | cache
Weather download: batch 78/100 ( 78.0%) | points 780/1000 | cache
Weather download: batch 79/100 ( 79.0%) | points 790/1000 | cache
Weather download: batch 80/100 ( 80.0%) | points 800/1000 | cache
Weather download: batch 81/100 ( 81.0%) | points 810/1000 | cache
Weather download: batch 82/100 ( 82.0%) | points 820/1000 | cache
Weather download: batch 83/100 ( 83.0%) | points 830/1000 | cache
Weather download: batch 84/100 ( 84.0%) | points 840/1000 | cache
Weather download: batch 85/100 ( 85.0%) | points 850/1000 | cache
Weather download: batch 86/100 ( 86.0%) | points 860/1000 | cache
Weather download: batch 87/100 ( 87.0%) | points 870/1000 | cache
Weather download: batch 88/100 ( 88.0%) | points 880/1000 | cache
Weather download: batch 89/100 ( 89.0%) | points 890/1000 | cache
Weather download: batch 90/100 ( 90.0%) | points 900/1000 | cache
Weather download: batch 91/100 ( 91.0%) | points 910/1000 | cache
Weather download: batch 92/100 ( 92.0%) | points 920/1000 | cache
Weather download: batch 93/100 ( 93.0%) | points 930/1000 | cache
Weather download: batch 94/100 ( 94.0%) | points 940/1000 | cache
Weather download: batch 95/100 ( 95.0%) | points 950/1000 | cache
Weather download: batch 96/100 ( 96.0%) | points 960/1000 | cache
Weather download: batch 97/100 ( 97.0%) | points 970/1000 | cache
Weather download: batch 98/100 ( 98.0%) | points 980/1000 | cache
Weather download: batch 99/100 ( 99.0%) | points 990/1000 | cache
Weather download: batch 100/100 (100.0%) | points 1000/1000 | cache
```

	precipitation	temperature_2m	wind_speed_10m
time			
2024-01-01 00:00:00+00:00	0.0014	-7.5063	27.1967
2024-01-01 01:00:00+00:00	0.0021	-7.5743	27.1660
2024-01-01 02:00:00+00:00	0.0021	-7.4283	27.3894
2024-01-01 03:00:00+00:00	0.0021	-7.3288	27.5036
2024-01-01 04:00:00+00:00	0.0006	-7.2007	27.6741

```
Hourly observations: 8784
```

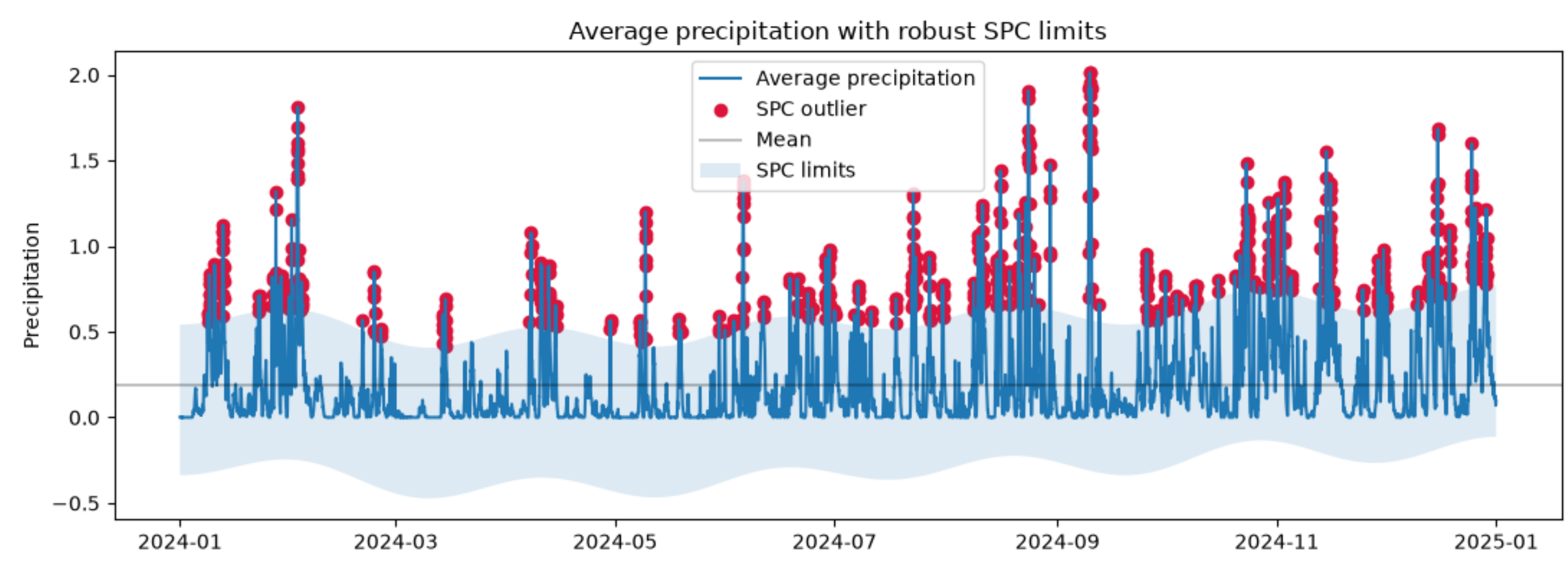
In [15]:
```
# 2.1 Publish prepared weather data to MongoDB
# The public Streamlit app reads these hourly averages from MongoDB instead
# of calling Open-Meteo for every visitor. The URI is read from local secrets.

# Publish the aggregated hourly weather data for the public Streamlit app.
import tomllib
from ind320_app.part3 import publish_weather_to_mongodb

secrets_path = PROJECT_ROOT / '.streamlit' / 'secrets.toml'
if not secrets_path.exists():
    raise FileNotFoundError('Create .streamlit/secrets.toml with [mongo].uri before publishing.')
with secrets_path.open('rb') as file:
    mongo_secrets = tomllib.load(file)
mongo_uri = mongo_secrets.get('mongo', {}).get('uri', '')
written = publish_weather_to_mongodb(
    weather_no3,
    region='NO3',
    mongo_uri=mongo_uri,
    collection='weather',
)
print(f'MongoDB weather upserts completed: {written:,}')
```

```
MongoDB weather upserts completed: 8,784
```

3. Precipitation: DCT high-pass filter and robust SPC

The seasonal component is reconstructed from the first DCT coefficients. Robust median/MAD limits are calculated from the seasonally adjusted values, while the plot deliberately shows only the original precipitation, limits and highlighted outliers.

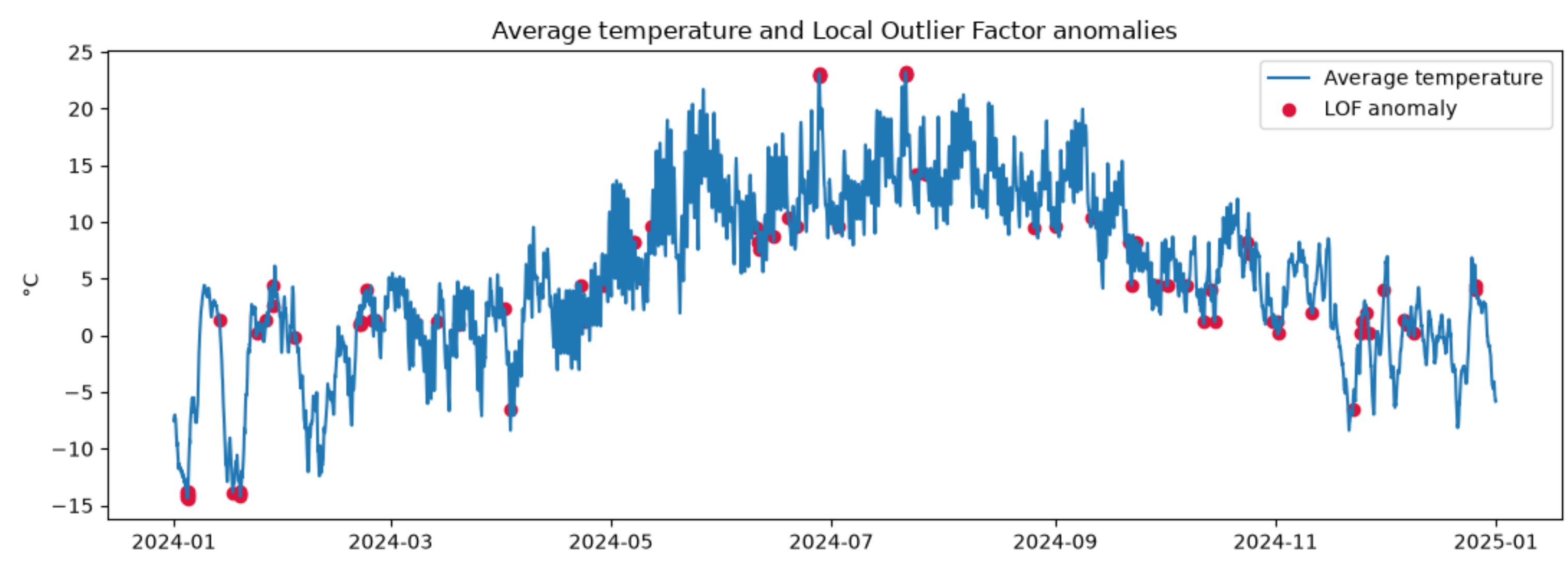
In [20]:
```
precipitation_figure, precipitation_summary = plot_spc_precipitation(
    weather_no3['precipitation'], cutoff=12, sigma=3.0
)
plt.show(precipitation_figure)
print('SPC outliers:', int(precipitation_summary['outlier'].sum()))
```



```
SPC outliers: 651
```

4. Temperature: Local Outlier Factor

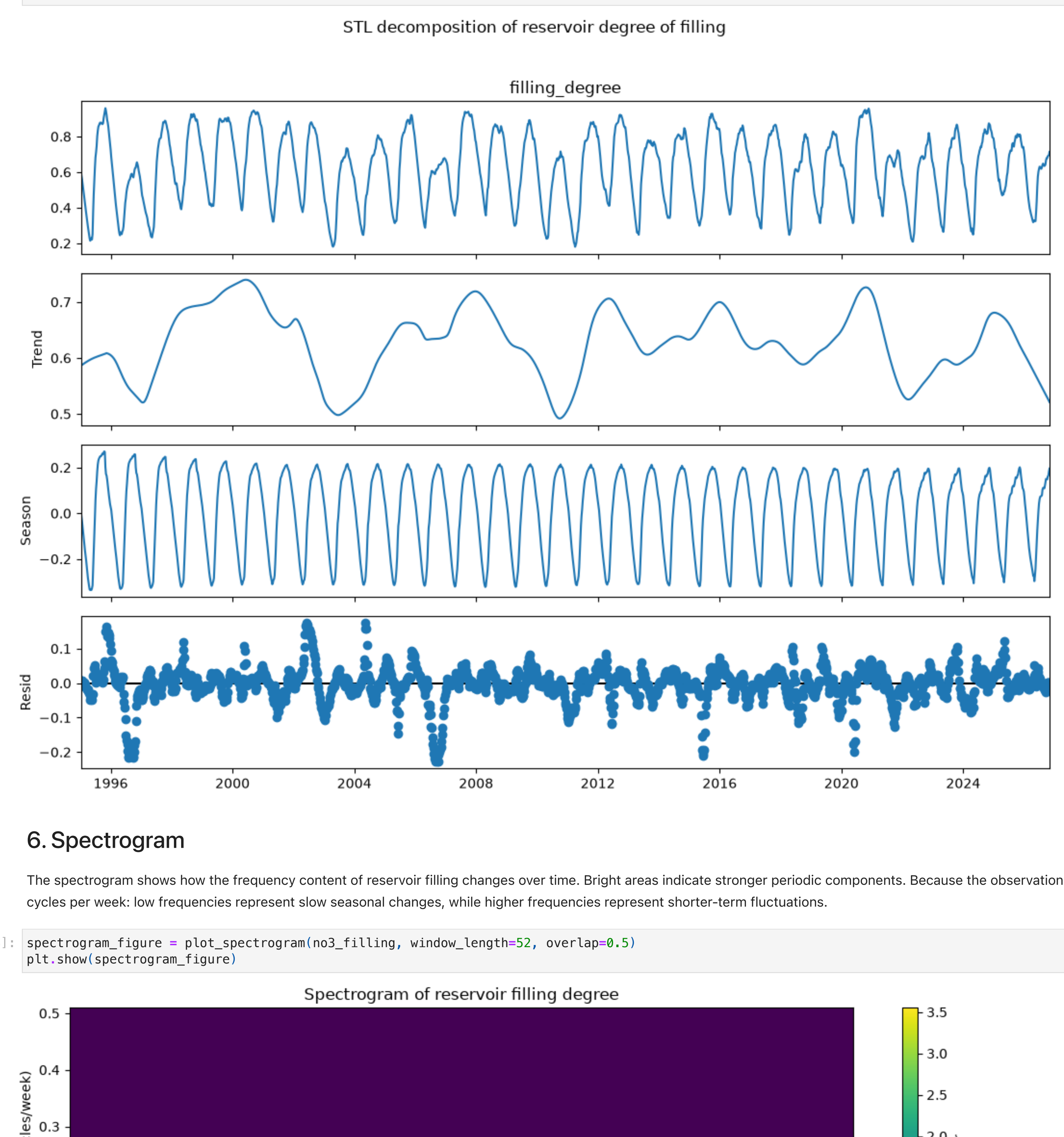
In [21]:
```
temperature_figure, temperature_summary = plot_lof_temperature(
    weather_no3['temperature_2m'], contamination=0.01
)
plt.show(temperature_figure)
print('LOF anomalies:', int(temperature_summary['anomaly'].sum()))
```



```
LOF anomalies: 88
```

5. STL decomposition of reservoir degree of filling

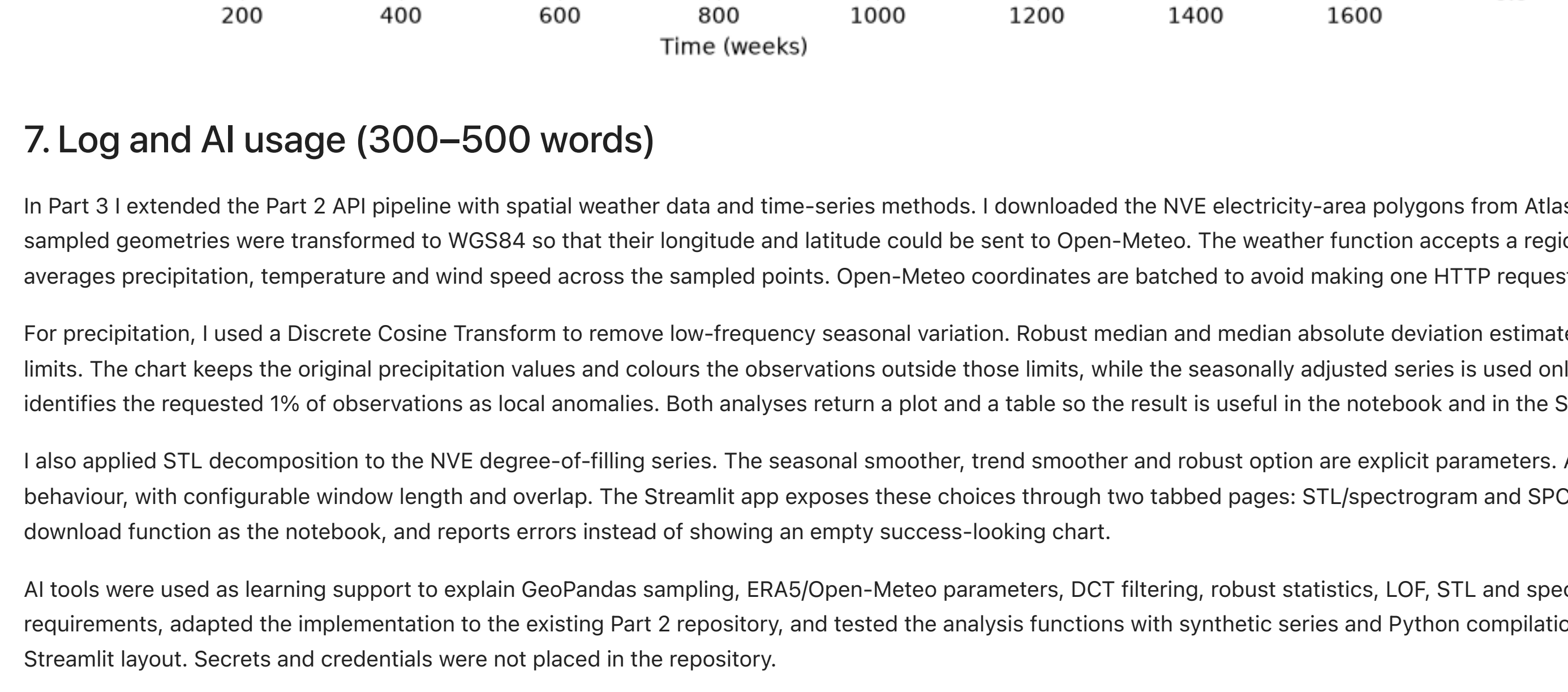
In [22]:
```
reservoirs = load_reservoir_data()
no3_filling = (
    reservoirs[(reservoirs['area_type'] == 'NO') & (reservoirs['area_number'] == 0)]
    .set_index('date')['filling_degree'].dropna()
)
stl_figure = plot_stl(no3_filling, seasonal=13, trend=None, robust=True)
plt.show(stl_figure)
```



6. Spectrogram

The spectrogram shows how the frequency content of reservoir filling changes over time. Bright areas indicate stronger periodic components. Because the observations are weekly, the frequency is measured in cycles per week: low frequencies represent slow seasonal changes, while higher frequencies represent shorter-term fluctuations.

In [23]:
```
spectrogram_figure = plot_spectrogram(no3_filling, window_length=52, overlap=0.5)
plt.show(spectrogram_figure)
```



7. Log and AI usage (300–500 words)

In Part 3 I extended the Part 2 API pipeline with spatial weather data and time-series methods. I downloaded the NVE electricity-area polygons from Atlas and sampled 1000 reproducible locations per region. The sampled geometries were transformed to WGS84 so that their longitude and latitude could be sent to Open-Meteo. The weather function accepts a region and interval, requests hourly ERA5 reanalysis data, and averages precipitation, temperature and wind speed across the sampled points. Open-Meteo coordinates are batched to avoid making one HTTP request per point.

For precipitation, I used a Discrete Cosine Transform to remove low-frequency seasonal variation. Robust median and median absolute deviation estimates from the full year define statistical process-control limits. The chart keeps the original precipitation values and colours the observations outside those limits, while the seasonally adjusted series is used only for classification. For temperature, Local Outlier Factor identifies the requested 1% of observations as local anomalies. Both analyses return a plot and a table so the result is useful in the notebook and in the Streamlit app.

I also applied STL decomposition to the NVE degree-of-filling series. The seasonal smoother, trend smoother and robust option are explicit parameters. A spectrogram provides a complementary view of periodic behaviour, with configurable window length and overlap. The Streamlit app exposes these choices through two tabbed pages: STL/spectrogram and SPC/LOF. The weather page uses the same region selector and download function as the notebook, and reports errors instead of showing an empty success-looking chart.

AI tools were used as learning support to explain GeoPandas sampling, ERA5/Open-Meteo parameters, DCT filtering, robust statistics, LOF, STL and spectrograms. I reviewed the suggestions against the course requirements, adapted the implementation to the existing Part 2 repository, and tested the analysis functions with synthetic series and Python compilation. AI was also used to improve documentation and Streamlit layout. Secrets and credentials were not placed in the repository.